Math 550 HW1 **Name: Hayden Floyd**

System Definitions

The objective of this mini-project assignment is to simulate the Stokes equations in 2D on a staggered grid. Below are the equations we will be solving:

$$\mu \nabla^2 \mathbf{u} - \nabla P = \mathbf{b}, \tag{1}$$

$$\nabla \cdot \mathbf{u} = \mathbf{0}. \tag{2}$$

To solve these, let's specify a few things.

1. $\mathbf{u}$ is the velocity vector of the fluid, which can be defined as $\mathbf{u} = [U(x, y), V(x, y)]$.
2. $U(x, y), V(x, y)$ are the x- and y-components of the velocity, respectively.
3. $P(x, y)$ is the pressure of the system. This can be thought of as a scalar function that serves as a Lagrange multiplier to enforce the divergence-free constraint.
4. $\mathbf{b}$ is the right hand side of our equation, where $\mathbf{b} = [f(x, y), g(x, y)]$.
5. The viscosity of the fluid, μ, will be constant. Specifically, let's set $\mu = 1$.

Let's also define our boundary conditions. We will assume:

1. periodic boundary conditions for U, V, and P on the left and right boundaries,
2. no boundary conditions for P on our top and bottom boundaries (why is this?),
3. Dirichlet boundary conditions for U in the y-direction, where $U = 0$ on our top and bottom boundaries,
4. and inflow/outflow (Dirichlet) conditions for V in the y direction, where $V = -3.5$ units on our top and bottom boundaries.

Figure 1 shows a cartoon of the grid as well as the boundary conditions and unknowns.

Simulation Solution Process

To solve these equations, we will discretize them on our staggered (MAC) grid and write the discrete unknowns and their boundary conditions as a sparse saddle point system. Thus, we can rewrite our Navier-Stokes equation as this following system:

$$\begin{bmatrix} \mu \mathcal{L}_u & 0 & -G_x \\ 0 & \mu \mathcal{L}_v & -G_y \\ D_x & D_y & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U} \\ \mathbf{V} \\ \mathbf{P} \end{bmatrix} = \begin{bmatrix} \mathbf{f}_{bc} \\ \mathbf{g}_{bc} \\ \mathbf{h}_{bc} \end{bmatrix},$$

where:

1. $\mathcal{L}_U$ is the Laplacian operator on the U grid,

2. $\mathcal{L}_V$ is the Laplacian operator on the V grid,

3. G_x is the x-component of the gradient operator on the P grid,

4. G_y is the y-component of the gradient operator on the P grid,

5. D_x is the x-component of the divergence operator on the U grid,

6. and D_y is the y-component of the divergence operator on the V grid.

1. **Problem 1: Solve Stokes**
   We will use the method of manufactured solutions to verify that our system is operating as expected. So, let's define analytical functions that align with our chosen boundary conditions:

$$U_{an}(x, y) = \sin(2\pi y)\sin(2\pi x) \tag{3}$$

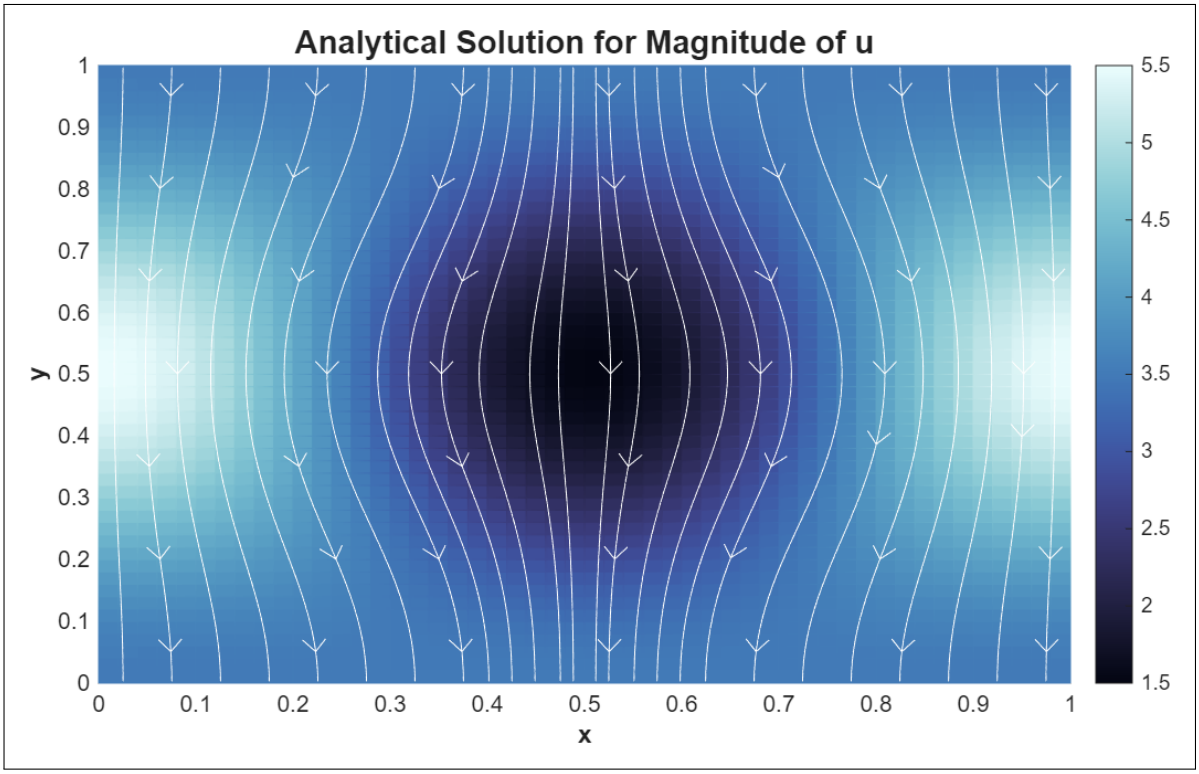
$$V_{an}(x, y) = -3.5 + \cos(2\pi x)[\cos(2\pi y) - 1] \tag{4}$$

$$P_{an}(x, y) = \sin(2\pi y)\cos(2\pi x) \tag{5}$$

   From these, we can define our right hand side to force our solutions to fit these equations and solve our system using a standard backslash technique (for $Ax = b, A\backslash b = x$). We can verify our solution by checking the magnitude of $\mathbf{u}$, which we denote as $||\mathbf{u}|| = \sqrt{U^2 + V^2}$. See Figure 2 for the solution plot.

   Note as well that P_{sol} is not uniquely determined, so you will have to subtract off the mean of P_{sol} from your solution when comparing to P_{an} in the next section.

Solution: (Numerical Solution being worked on currently)



2. **Problem 2: Error Analysis**
   Once the system has been solved, we can define the error of the solution, since we know that the solution should fit the forms described by equations 3, 4, and 5. To do this, iterate over an increasing number of grid points ($Nx = Ny$) and show that we see second-order error decreases in our numerical solution (U_{num}) compared to our analytical solution (U_{an}). Refer to Figure 3 for an example of the expected error plots. We will calculate error using the following norm equation:

$$\text{err}_U = \frac{||U_{num} - U_{an}||_2}{||U_{an}||_2} \tag{6}$$

Solution:

3. **Problem 3: Charlie's Rule**
   Charlie Peskin (the inventor of the IBM) says that you should never just stop once you have a cool code, **the best part is to play around with it.** So do something fun with your code and show it off! The following are a few ideas:

   1. The 'Stokes Darcy' equations are simple modification of the Stokes equations that let us model flow through porous media. Read about these equations and try simulating flow through some kind of porous microstructure.

2. Try combining your code with the Immersed Boundary (IB) code from class to look at fluid flow with point like particles.
3. Your code will be much more flexible and robust if you make it work (and test it!) for several different kinds of boundary conditions.
4. Using a direct solver ('backslash' in MATLAB) works great for this example, but it won't for bigger problems. Explore other linear solvers e.g. GMRES to speed up your code.

Solution: